❹ If the model is valid, calculates the result

❺ Returns the result directly

Let's first consider the happy path, when the controller receives a valid request. The following listing shows that you can create an instance of the API controller, call an action method, and receive an `ActionResult<T>` response.

Listing 36.5 A simple API controller unit test

```
public class CurrencyControllerTest
{
    [Fact]
    public void Convert_ReturnsValue()
    {
        var controller = new CurrencyController();                ❶
        var model = new InputModel                                ❶
        {                                                         ❶
            Value = 1,                                            ❶
            ExchangeRate = 3,                                     ❶
            DecimalPlaces = 2,                                    ❶
        };                                                        ❶

        ActionResult<decimal> result = controller.Convert(model); ❷
        Assert.NotNull(result);                                   ❸
    }
}
```

❶ Creates an instance of the ConvertController to test and a model to send to the API

❷ Invokes the ConvertToGbp method and captures the value returned

❸ Asserts that the IActionResult is not null

An important point to note here is that you're testing only the return value of the action, the `ActionResult<T>`, not the response that's sent back to the user. The process of serializing the result to the response is handled by the Model-View-Controller (MVC) formatter infrastructure, as you saw in chapter 9, not by the controller.